

Supabase

Supabase

Supabase is the Postgres development platform.

Start your project with a Postgres database, Authentication, instant APIs, Edge Functions, Realtime subscriptions, Storage, and Vector embeddings.

It is open-source 🙌

And not part of American Big Tech!

Supabase

Postgres is a powerful, open-source object-relational database system known for its reliability, feature robustness, and performance.

It supports both SQL (relational) and JSON (non-relational) data, making it suitable for complex data workloads.

So everything you have learned about SQL database can be applied to Supabase 🙌

Supabase

behu-kea's Org

FREE

Android app

main

PRODUCTION

Connect

Feedback

Search...

⌘ K

?

💡

📄

🔍

🌱

Project Overview

Table Editor

SQL Editor

Database

Authentication

Storage

Edge Functions

Realtime

Advisors

Observability

Logs

API Docs

Integrations

Project Settings

Android app

NANO

Tables

1

Functions

0

Replicas

0

Project Status

Last 60 minutes

Statistics for last 60 minutes

Database

REST Requests

0

No data to show
It may take up to 24 hours for data to refresh

Auth

Auth Requests

0

No data to show
It may take up to 24 hours for data to refresh

Storage

Storage Requests

0

No data to show
It may take up to 24 hours for data to refresh

Realtime

Realtime Requests

0

No data to show
It may take up to 24 hours for data to refresh

2 issues need attention

SECURITY

2

PERFORMANCE

SLOW QUERIES

Supabase - tables

⚡

/ behu-kea's Org FREE

/ Android app

/ main PRODUCTION

Connect

🏠

📊

🔍

📁

📄

🔔

🔧

🔧

🔧

🔧

⚙️

Table Editor

schema public

+ New table

🔍 Search tables...

🌐 instruments

public.instruments

Filter

Sort

Insert

	🔑 id int8	name text	
☐	1	violin	
☐	3	cello	
☐	5	guitar	
☐	6	guitar	
☐	7	Drums	
☐	8	Drums	
☐	9	Drums	

Supabase - SQL editor

SQL Editor

⌵ +

> SHARED

> FAVORITES

⌵ PRIVATE (2)

SQL Public read policy for instrume...

SQL Instruments table with row-lev...

⌵ COMMUNITY

≡ Templates

≡ Quickstarts

SQL Instruments table with row-level security

SQL Public read policy for instruments table

+

```
1 insert into instruments (name)
2 values
3   ('violin'),
4   ('viola'),
5   ('cello');
```

Results

Explain

Chart

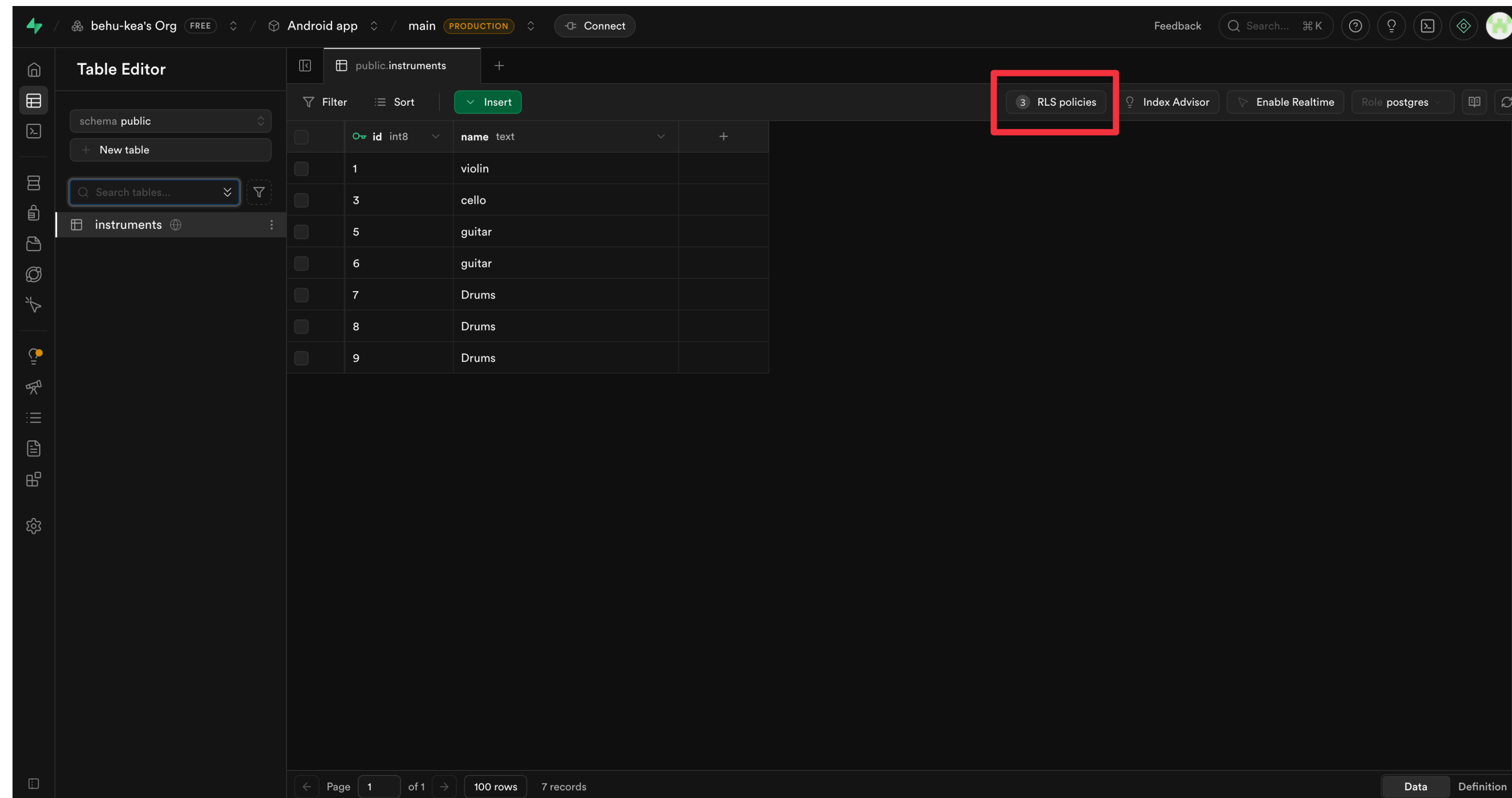
Click Run to execute your query.

Supabase - RLS

Secure your data using
Postgres Row Level Security -
RLS

Each policy is attached to a
table, and the policy is
executed every time a table is
accessed!

You can just think of RLS as
adding a where clause to every
query



Supabase - RLS

For each command (Select, Insert, Delete and Update) you specify who can access a specific row!

Here are 3 rules!

Policies

Manage Row Level Security policies for your tables

schema public

instruments

instruments

Disable RLS

Create policy

NAME	COMMAND	APPLIED TO	
Everyone can delete	DELETE	public	
Everyone can insert	INSERT	public	
Everyone can read	SELECT	anon	

Supabase - RLS

This RLS is doing the following:

- Create a new policy called “public read guitar”
- Apply the RLS to the instruments table
- Permissive means that if the RLS is true the user can get access no matter which other RLS there is on the row!
- The command is Select
- The role is public, meaning everyone
- The query that decides if a user can see a role is `name = 'guitar'`. So a user can only see the guitars and no other instruments

The screenshot shows the Supabase dashboard interface for creating a new Row Level Security (RLS) policy. The form is titled "Create a new Row Level Security policy". It contains the following fields and options:

- Policy Name:** `public read guitar`
- Table on clause:** `public.instruments`
- Policy Behavior as clause:** `Permissive`
- Policy Command for clause:** `SELECT` (selected), `INSERT`, `UPDATE`, `DELETE`, `ALL`
- RLS name:** Defaults to all (public) roles if none selected

Below the form, the SQL command is displayed in a code editor:

```
1 create policy "public read guitar"
2 on "public"."instruments"
3 as PERMISSIVE
4 for SELECT
5 to public
6 using (
7   name = 'guitar'
8 );
```

Annotations with red arrows point to specific parts of the form and the SQL code:

- Policies combined with OR:** Points to the "Policy Name" field.
- Target role:** Points to the "RLS name" field.
- Table to apply RLS:** Points to the "Table on clause" field.
- Command:** Points to the "Policy Command for clause" field.
- query to run for each row:** Points to the `using (` clause in the SQL code.

Supabase - RLS

A classic RLS rule is

using(

(`select` auth.uid()) = user_id

)

This means the user can only see the entities that user has created. The user_id is a row in a table that refers to a users table

But this needs authentication though!

```
USE OPTIONS ABOVE TO EDIT
alter policy "Users can view their own profile."
on "public"."profiles"
to public
using (
  (auth.uid() = user_id)
);
```

 id uuid	 user_id uuid	full_name text	school text
---	--	----------------	-------------

Repository - design pattern

The idea is you create a class that has all the operations you need to interact with the database

getCustomer, deleteCustomer, updateCustomer, etc

This is the repository. It is an abstraction.

The repository pattern is a way to abstract out the implementation details of how your application communicates with your persistence layer (database).

```
1 Usage
class InstrumentRepository {
    1 Usage
    suspend fun deleteInstrument(id: Int) {
        supabase
        .from( table = "instruments")
        .delete {
            filter {
                eq( column = "id", value = id)
            }
        }
    }

    1 Usage
    suspend fun createNewInstrument(instrument: Instrument) {
        supabase
        .from( table = "instruments")
        .insert( value = instrument)
    }

    2 Usages
    suspend fun getInstruments(): List<Instrument> {
        return supabase
        .from( table = "instruments")
        .select()
        .decodeList<Instrument>()
    }
}
```

Repository - design pattern

```
Button(onClick = {
    notesViewModel.createInstrument()
}) {
    Text(text = "Create new instrument")
}
```

4 Usages

```
val instrumentRepository = InstrumentRepository()
```

2 Usages

```
class InstrumentViewModel: ViewModel() {
```

3 Usages

```
var instruments by mutableStateOf<List<Instrument>>(value = listOf())
```

```
init {
```

```
    viewModelScope.launch {
```

```
        instruments = instrumentRepository.getInstruments()
```

```
    }
```

```
}  
|
```

1 Usage

```
fun createInstrument() {
```

```
    viewModelScope.launch {
```

```
        val newInstrument = Instrument(name = "Drums");
```

```
        instrumentRepository.createNewInstrument(newInstrument)
```

```
    }
```

```
}
```